

Simplitest — Product Requirements Document

A Test Management Platform for SimpliEd QA

Status	Final v3.0
Prepared by	Ansa Fatima, QA Engineering
Date	September 11, 2026
Audience	Engineering & QA management

This revision reworks **Reporting & Analytics** end to end — a redesigned Dashboard, a filterable Stability report (Period/Sprint, Portal, Module, Feature), and a new **Cycle History** report that replaces the old Execution and Release reports — and documents real file **attachments** on test cases, and **tester attribution** on both test runs and quick logs. Every other section is carried forward for context.

1. Executive summary

Simplitest is a purpose-built test management platform for SimpliEd's QA organization. It replaces spreadsheets and disconnected trackers with a single workspace for organizing test cases, running both structured regression cycles and lightweight ad hoc checks, and reporting on release readiness and long-term module stability across every SimpliEd product surface — Admin Portal, Teacher/Parent/Student apps, QR Attendance, and any portal added later.

2. Problem statement

- Test cases and results were scattered across spreadsheets and documents, with no single source of truth for what had been tested or what passed.
- There was no consistent way to see a module's pass/fail history over time, so instability crept in quietly instead of showing up as an early warning before release.
- Managers had no self-serve visibility into QA coverage, tester workload, or release health — status required manually pinging QA.
- Ad hoc, same-day testing was too heavy to log formally, so it often went untracked entirely.

- Onboarding a new tester meant explaining ad hoc conventions instead of pointing them at a shared structure.

3. Goals

- **Single source of truth** — every portal/app has one structured home for its test cases (Portal → Module → Suite → Test Case).
- **Two speeds of testing** — support full case-by-case regression cycles and 30-second quick logs, so lightweight testing gets recorded instead of skipped.
- **Early warning on instability** — surface which modules are trending toward instability before they become a release incident.
- **Self-serve reporting** — give managers and leads dashboard, stability, and cycle-history reporting without asking QA for manual updates.
- **Low-friction team admin** — invites, roles, and account recovery that don't depend on email infrastructure.

4. Non-goals (out of scope for this version)

- Automated test execution or CI test-runner integration — today's runs are logged manually, whether or not the testing itself was automated.
- Evidence attachments on a test **run's result** (e.g. a screenshot of a specific failure) — shown as "Coming soon" in the UI, not yet built. (Test **cases** themselves do support small file attachments today — see §7.1.)
- Replacing a defect tracker — issue counts and severities are tracked, but there's no ticket workflow; a ticket-link field points out to an external tracker.
- Real-time collaboration beyond per-run notes (no comment threads or @mentions).
- A native mobile app — the web app is responsive, not a packaged app.
- A dedicated tester-performance leaderboard (pass rate ranked per tester) — Cycle History's Tester filter and per-cycle attribution cover "who ran what," but ranking testers against each other stays out of scope.

5. Target users

- **QA Tester** — logs test results (case-based or quick log), files issues, and tracks their own execution history.

- **QA Manager / Lead** — plans test cycles, defines scope, reviews reports, and signs off releases based on stability data.
- **Developer** — checks failures relevant to their module and the stability trend of an area before making changes.
- **Super Admin** — manages workspace membership, roles, and account recovery for the whole team.
- **Viewer / Stakeholder** — read-only access to reports and dashboards to check release health without a QA background.

6. Product structure

Test content is organized in a fixed hierarchy so every portal has a predictable home for its cases:

Workspace → Portal (app/product) → Module (feature area) → Suite (nested folder, any depth) → Test Case

A test cycle then runs against that structure in one of two modes: **Case-based** (a scoped regression run, executed case by case) or **Manual quick log** (a single aggregate verdict logged in seconds). Both are detailed in full below.

7. Core features & requirements

7.1 Test case management

Test case management is where every portal's test content lives — one structured, searchable home instead of scattered spreadsheets, with the bulk tooling and import path needed to actually populate and maintain it at scale.

- **Structured hierarchy, unlimited depth.** Every case attaches to exactly one of Portal, Module, or Suite — and Suites can nest arbitrarily, so a team can model "Admin Portal → Academic → Dashboard → Widgets" as deep as it actually needs to go, not just a fixed two levels.
- **Full case detail, not just a title.** Title, subtitle, description, preconditions, a structured step-by-step checklist (steps are stored as an ordered list, not a single text blob, so each step can be added, reordered, or removed independently), expected result, and an optional owner (a real workspace member, shown as an avatar in the list).
- **A closed, consistent vocabulary:**

- Priority — High / Medium / Low
- Severity — Critical / Major / Minor
- Type — Functional / Regression / Smoke / Sanity / UI / API
- Status — Active / Draft / Archived
- **Search and filter kept deliberately minimal** — filtering is limited to Priority and Type specifically so every filter option maps to something the tester can actually see in the table, rather than a filter panel that's wider than the data it's filtering.
- **Bulk actions** — multi-select bulk field edit, bulk move between hierarchy locations, duplicate, and delete. This is the exact tooling that made the Mobile App → Teacher/Parent/Student App portal restructuring possible without a manual, error-prone case-by-case migration.
- **CSV import** for bringing in existing suites wholesale — this is how the current ~3,400-case SimpliEd test suite was migrated into Simplitest in the first place, rather than retyped by hand.
- **Stable external references** — every case gets a permanent, sequential case number, so a case can be cited in a bug report or a conversation ("TC-142 is failing") without ambiguity.
- **Attachments.** Up to 5 small files per case (a screenshot, a spec snippet) — stored inline with the case, no separate file-storage service to run — capped at 5 MB per file.

7.2 Test execution (test runs)

Test execution covers everything to do with actually running tests and recording results, at two different speeds depending on how heavy the testing needs to be. Every test cycle runs in one of two modes:

Case-based cycles — full regression execution:

- Scope picker: **All** cases, a whole **Portal**, a **Module**, a **Suite**, or a hand-picked **Custom** selection — so a cycle can be as broad as "everything" or as narrow as "these 12 cases I just touched."
- Each in-scope case becomes its own run, scored **Passed / Failed / Blocked / Skipped / Not Run**, with notes, who executed it, and when.
- Live progress while the cycle is open — executed vs. remaining, and a Pass/Fail/Blocked breakdown, so a lead can see cycle health without opening every row.

Manual quick logs — the fast path:

- A single aggregate entry against a Module or Suite, capturing total issue count plus a Critical/Major/Minor severity breakdown, environment (Production/QA/Staging/Dev), platform (Android/iPhone/Web/Desktop/All), app version, a category

(Stability/Regression/Functional/UI/Performance), and an optional link to an external ticket — the whole thing takes seconds, which is the entire point.

- A quick log can be created against a module or suite by name even before that part of the hierarchy formally exists in the tool, so testers are never blocked from logging just because the structure hasn't caught up yet.

Retesting is an edit to the same record, not a new cycle:

- A quick log carries its own **Done** and **Remaining** issue counts, editable directly on that same cycle as fixes land and get re-verified — there is no separate "retest" object and nothing new gets created. Reopening a quick log and updating Done/Remaining is the retest.
- These counts are what feed the Stability report's partial-credit scoring (§7.3.2).

Findability, once logged:

- Every run — case-based or quick log — is sortable and filterable by date (newest first by default), portal, module, and status, and every report that references a run links straight back to it.
- **Tester attribution.** Both a case-based run and a quick log record who performed it — attached automatically from the signed-in session, never a manual field to fill in. This is what the Cycle History report's Tester filter matches against; a run or log from before attribution existed simply won't match any tester filter.

7.3 Reporting & analytics

Simplitest's reporting surfaces are the **Dashboard** (the home-screen overview, seen first on every visit) and, under **Reports**, two dedicated reports: **Stability** and **Cycle History**. Two earlier reports — Execution and Release — were retired; their content is now fully covered between Stability (is this area getting better or worse) and Cycle History (a filterable log of every cycle that's ever run), so there's one place to check per question instead of three overlapping ones.

Both Stability and Cycle History share the same scoping filters, so narrowing down "which module, which sprint" works identically wherever you are:

- **Period** — Today / Last 7 days / Last 30 days / Sprint / All time.
- **Sprint** is a fixed, calendar-aligned 2-week block (not a rolling 14-day window), anchored to the team's real sprint start date — so "this sprint" means the same calendar dates for everyone. A stepper pages backward through prior sprints one at a time; the current sprint stays open-ended (runs to now) while a

past sprint gets its real closing date.

- **Portal → Module → Feature (Suite)** — the same cascading scope picker used everywhere else in the product. Narrowing to a module or feature automatically hides portals/modules that have nothing in scope, rather than padding the view with empty rows.

7.3.1 Dashboard

The home screen answers "what's going on right now" at a glance:

- A greeting banner naming how many test runs are currently in progress and how many failures are open, with a one-click link into Test Runs.
- Four 30-day stat cards — **Passed**, **Failed**, **Blocked**, and current **Open failures** — each with its trend against the prior 30-day window.
- An **Active test run** card, shown whenever one is open: scope, a live progress bar, pass rate, and a Continue button straight into the run.
- An 8-week execution trend chart and a Pass/Fail/Blocked donut summary.
- A **Coverage by module** panel — real per-module pass rates, worst-covered first.
- A **Recent test runs** table — run name, progress, result (Pass / Fail / Active), and who ran it (tester name, or "Unattributed" for runs and logs from before tester attribution existed).

7.3.2 Stability report

The Stability report answers "is this module getting better or worse," by blending every recorded result for a module or suite into one rolling pass rate with a trend, rather than showing any single run in isolation.

- **Two data sources, blended per module/suite.** Case-based test runs with a Passed/Failed result, and Manual quick logs, are combined into one rolling data set per Module and per Suite. Blocked/Skipped/Not Run results are excluded — they aren't a verdict on stability either way.
- **Rollups follow the hierarchy.** A Suite's numbers include every nested child suite underneath it, at any depth. A Module's numbers include its own directly-attached cases/logs *plus* every suite under it.
- **Pass/fail rule:**
 - *Untracked* quick logs (Done and Remaining never touched) — Pass only if the original issue count was zero.
 - *Tracked* quick logs (Done and/or Remaining have been recorded) — Pass whenever the live Remaining count is zero, regardless of how many issues were originally found. This is what makes a fully-resolved cycle correctly read as a pass.
- **Partial-credit scoring.** Every data point carries a 0–1 score, not just a binary pass/fail. A tracked quick log scores $\text{done} \div (\text{done} + \text{remaining})$ — 6 of 8 resolved scores **0.75**. A case-based run scores 1

(Passed) or 0 (Failed). The report's headline **pass rate is the average score** across all data points, not a raw count of full passes — so a module full of half-resolved quick logs correctly reads as meaningfully better than a flat 0%. The separate Passed/Failed counts shown alongside stay binary, for anyone who wants the raw tally instead.

- **Classification thresholds** — pass rate $\geq 90\%$ → **Stable**, $\geq 70\%$ → **At Risk**, otherwise → **Unstable**. A module or suite with zero data points is labeled **No data**, kept distinct from "bad" so an untested area doesn't visually masquerade as a risk — and it sorts to the bottom of the risk-ranked list rather than looking alarming by default.
- **Trend indicator**. A module's data points are split into an earlier and a later half by timestamp (at least 4 points are required to say anything meaningful), and the average score of each half is compared: **Up** (≥ 5 -point improvement), **Down** (≥ 5 -point decline), or **Flat**.
- **Overview panel** — a coverage-by-module bar list and a pass/fail donut, both computed from whatever the active Period/Portal/Module/Feature scope resolves to, so the summary at the top and the drill-down list below it can never silently disagree.
- **Drill-down panel**. Clicking any module or suite opens its own pass-rate trend chart, plus the *full* list of every underlying cycle — case-based runs and quick logs together, newest first, not filtered to failures only — each tagged Pass/Fail with its own detail line (for example, *"Fail · 3 of 8 issues still open"*), and each linking straight through to that specific run or quick log.
- **Export and sharing** — CSV/PDF export and a shareable link.

7.3.3 Cycle History report

A flat, filterable log of every cycle — case-based test run or quick log — answering "how many cycles ran against this module, and by whom" directly, instead of opening each cycle in turn to check.

- The same Period/Portal/Module/Feature filters as Stability, plus a **Tester** filter — matches whoever executed a case-based run, or logged a quick log. A run or log from before tester attribution existed won't match any tester filter, since there's nothing honest to attribute it to.
- A KPI row: total cycles, total issues logged, quick-log count, test-run count.
- A **cycles per module** bar list, and the full row-level table — cycle name, type (quick log / test run), Module > Feature, tester, issue count, and result (Active / Pass / Fail) — each row linking through to the underlying run.
- CSV export.

7.4 Team & access management

- Role-based access across five roles — Super Admin, QA Manager, Tester, Developer, Viewer — enforced at both the workspace level and, per project, via each member's own workspace role (not a single

global label), so permissions stay correct even as someone's role changes in one workspace but not another.

- Invite-link onboarding — no email server required.
- Admin-mediated password reset — a Super Admin generates a one-time reset link for a locked-out teammate, without ever seeing or setting their password directly.
- A workspace's creator, or anyone currently holding Super Admin in that workspace, always retains the ability to manage their own role — so stepping down temporarily can never turn into a permanent lockout from administering a workspace they own.

8. Success metrics

- Share of releases with a completed case-based regression cycle before go-live.
- Modules that reach "At Risk" before they reach "Unstable" — i.e., the trend gives a real early warning instead of a same-day surprise.
- Time from "testing needed" to "logged" for ad hoc work, as a proxy for quick-log adoption.
- Manager self-serve report views vs. manual status-update requests sent to QA.

9. Technical overview

Built on Next.js 14 (App Router) with TypeScript and Tailwind CSS, backed by PostgreSQL via Prisma. One codebase serves both the web application and its API. Deployment is continuous: pushes to the main branch run through GitHub Actions and deploy automatically via CapRover. The interface was restyled this cycle around a single indigo brand color driving every button, link, and highlight from one shared token, plus a slimmed icon-rail-and-topbar navigation layout.

10. Roadmap / future considerations

- Evidence attachments on a test run's **result** (e.g. a screenshot of a specific failure) — test cases themselves already support attachments (§7.1).
- CI / automated test-runner integration.
- Native mobile app or installable PWA.
- Two-way defect-tracker integration.

Appendix A: Feature status summary

Area	Status
Test case management	Live — hierarchy, bulk edit/move, search & filter, CSV import, small file attachments
Case-based test cycles	Live — scoped execution (all / portal / module / suite / custom pick), tester attribution
Manual quick logs	Live — lightweight aggregate logging with editable Done/Remaining tracking, tester attribution
Dashboard	Live — 30-day Passed/Failed/Blocked KPIs, active-run card, execution trend, coverage by module, recent runs
Stability report	Live — Period/Sprint + Portal/Module/Feature filters, blended pass rate with partial credit, trend, drill-down
Cycle History report	Live — every cycle in one filterable table (adds a Tester filter), cycles-per-module rollup, CSV export
Execution report	Retired — merged into Stability + Cycle History
Release report	Retired — merged into Stability + Cycle History
Team & role management	Live — invite links, 5-tier roles scoped per workspace, admin-mediated password reset
Evidence attachments on run results	Planned — placeholder in UI today
CI / automated test-runner integration	Not started
Two-way defect-tracker sync	Not started